

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CƠ KHÍ - BỘ MÔN CƠ ĐIỆN TỬ



NHẬP MÔN THỊ GIÁC MÁY TÍNH
EXERCISE 3

LỚP TN01 - HK 251

GVHD: PGS.TS. LÊ ĐỨC HẠNH

SINH VIÊN THỰC HIỆN:

STT	Họ và tên	MSSV
1	Võ Hữu Dư	2210604
2	Đào Trọng Chân	2210350
3	Dương Quang Duy	2210497
4	Trần Quang Đạo	2210647

TP.HCM, Ngày 28 tháng 11 năm 2025

Mục lục

1	Question 1	3
1.1	Tìm tọa độ, diện tích, chu vi, góc (độ) của hình sử dụng image moment và lập trình so sánh với hàm OpenCV	3
1.1.1	Tính tay sử dụng image moment	3
1.1.2	Lập trình kiểm tra và so sánh với kết quả sử dụng hàm OpenCV	5
2	Question 2	7
2.1	Tìm vị trí các máy bay trong hình bằng phương pháp genral hough transform tự lập trình	7
2.1.1	Các bước thực hiện	7
2.1.2	Kết quả thu được	11
2.2	Tìm vị trí các máy bay trong hình bằng phương pháp genral hough transform của OpenCV	17
2.2.1	Các bước thực hiện	17
2.2.2	Kết quả thu được	18
2.3	Nhận xét	22

Danh sách bảng

Danh sách hình vẽ

1.1	Kết quả lập trình tính toán các thông số	5
1.2	Kết quả sử dụng hàm OpenCV để kiểm tra	6
2.1	Các Template của các máy bay trong hình	7
2.2	Template 1	11
2.3	Kết quả tìm kiếm máy bay trong hình với Template 1	12
2.4	Template 2	12
2.5	Kết quả tìm kiếm máy bay trong hình với Template 2	13
2.6	Template 3	13
2.7	Kết quả tìm kiếm máy bay trong hình với Template 3	14
2.8	Template 4	14
2.9	Kết quả tìm kiếm máy bay trong hình với Template 4	15
2.10	Template 5	15
2.11	Kết quả tìm kiếm máy bay trong hình với Template 5	16
2.12	Template 6	16
2.13	Kết quả tìm kiếm máy bay trong hình với Template 6	17
2.14	Kết quả tìm kiếm máy bay trong hình với Template 1 bằng OpenCV GHT Ballard	19
2.15	Kết quả tìm kiếm máy bay trong hình với Template 2 bằng OpenCV GHT Ballard	20
2.16	Kết quả tìm kiếm máy bay trong hình với Template 3 bằng OpenCV GHT Ballard	20
2.17	Kết quả tìm kiếm máy bay trong hình với Template 4 bằng OpenCV GHT Ballard	21



2.18	Kết quả tìm kiếm máy bay trong hình với Template 5 bằng OpenCV GHT	
	Ballard	21
2.19	Kết quả tìm kiếm máy bay trong hình với Template 6 bằng OpenCV GHT	
	Ballard	22

1 Question 1

1.1 Tìm tọa độ, diện tích, chu vi, góc (độ) của hình sử dụng image moment và lập trình so sánh với hàm OpenCV

1.1.1 Tính tay sử dụng image moment

Hình có ma trận sau:

0	0	0	0	0	0	0
0	0	0	255	0	0	0
0	0	255	255	255	0	0
0	255	255	255	255	255	0
0	255	255	255	255	255	0
255	255	0	0	0	255	255
0	0	0	0	0	0	0

- Tọa độ tâm của hình và diện tích

Công thức tổng quát moment của ảnh

$$M_{p,q} = \sum_i \sum_j i^p j^q I(i,j) \quad (1.1)$$

Tọa độ tâm của ảnh được tính bằng công thức

$$x_c = \frac{M_{1,0}}{M_{0,0}} \quad (1.2)$$

$$y_c = \frac{M_{0,1}}{M_{0,0}} \quad (1.3)$$

Ta có

$$M_{0,0} = \sum_i \sum_j I(i,j) = 1 \times 18 = 18$$

$$\begin{aligned} M_{1,0} &= \sum_i \sum_j i \cdot I(i,j) \\ &= 1 \times (0 \times 1 + 1 \times 3 + 2 \times 3 + 3 \times 4 + 4 \times 3 + 5 \times 3 + 6 \times 1) = 54 \end{aligned}$$

$$\begin{aligned} M_{0,1} &= \sum_i \sum_j j \cdot I(i,j) \\ &= 1 \times (0 \times 0 + 1 \times 1 + 2 \times 3 + 3 \times 5 + 4 \times 5 + 5 \times 4 + 6 \times 0) = 62 \end{aligned}$$

Suy ra

$$S = M_{0,0} = 18$$

$$x_c = \frac{54}{18} = 3$$

$$y_c = \frac{62}{18} = 3.4444 \approx 3$$

0	0	0	0	0	0	0
0	0	0	b	0	0	0
0	0	b	0	b	0	0
0	c	0	0	0	c	0
0	c	c	a	c	c	0
c	c	0	0	0	c	c
0	0	0	0	0	0	0

• **Chu vi**

Contour của hình và loại pixel thể hiện trong hình
Chu vi của hình được tính bằng

$$P = n(a) + n(b)\sqrt{2} + n(c)\frac{1 + \sqrt{2}}{2}$$

Trong đó $n(a)$, $n(b)$, $n(c)$ lần lượt là số lượng pixel loại a, b, c.
Vậy ta có

$$P = 1 + 3 \times \sqrt{2} + 10 \times \frac{1 + \sqrt{2}}{2} \approx 17.3137$$

• **Góc của hình (độ)**

Góc xoay của hình được tính bằng công thức

$$\theta = \frac{1}{2} \arctan \left(\frac{2\mu'_{11}}{\mu'_{20} - \mu'_{02}} \right) \quad (1.4)$$

Trong đó

$$\begin{aligned} \mu'_{11} &= \frac{M_{1,1}}{M_{0,0}} - x_c y_c \\ \mu'_{20} &= \frac{M_{2,0}}{M_{0,0}} - x_c^2 \\ \mu'_{02} &= \frac{M_{0,2}}{M_{0,0}} - y_c^2 \end{aligned}$$

Lần lượt tính các moment cần thiết

$$\begin{aligned} M_{1,1} &= \sum_i \sum_j ijI(i, j) = 0 + 1 \times (3 + 4 + 5) + 2 \times (2 + 3 + 4) + 3 \times (1 + 2 + 3 + 4) \\ &\quad + 4 \times (2 + 3 + 4) + 5 \times (3 + 4 + 5) + 6 \times 5 = 186 \end{aligned}$$

$$M_{2,0} = \sum_i \sum_j i^2 I(i, j) = 0 + 1^2 \times 3 + 2^2 \times 3 + 3^2 \times 4 + 4^2 \times 3 + 5^2 \times 3 + 6^2 \times 1 = 210$$

$$M_{0,2} = \sum_i \sum_j j^2 I(i, j) = 0 + 1^2 \times 1 + 2^2 \times 3 + 3^2 \times 5 + 4^2 \times 5 + 5^2 \times 4 + 6^2 \times 0 = 238$$

Suy ra

$$\begin{aligned} \mu'_{11} &= \frac{186}{18} - 3 \times \frac{62}{18} = 0 \\ \mu'_{20} &= \frac{210}{18} - 3^2 = \frac{8}{3} \approx 2.67 \\ \mu'_{02} &= \frac{238}{18} - \left(\frac{62}{18} \right)^2 = \frac{110}{18} \approx 1.358 \end{aligned}$$

Thay vào công thức 1.4 ta có

$$\theta = \frac{1}{2} \arctan \left(\frac{2 \times 0}{2.67 - 1.358} \right) = 0^\circ$$

• Kết luận

Các giá trị tính toán tay theo yêu cầu đề bài như sau:

- Tọa độ tâm: $(3, 3.444) \approx (3, 3)$
- Diện tích: 18 pixel
- Chu vi: ≈ 17.3137
- Góc: 0°

1.1.2 Lập trình kiểm tra và so sánh với kết quả sử dụng hàm OpenCV

Đoạn code lập trình kiểm tra kết quả tính toán

```
1 ys, xs = np.where(img == 255)
2 M00 = len(xs)
3 M10 = np.sum(xs)
4 M01 = np.sum(ys)
5 M11 = np.sum(xs * ys)
6 M20 = np.sum(xs**2)
7 M02 = np.sum(ys**2)
8 x_centroid = M10 / M00
9 y_centroid = M01 / M00
10 u11 = M11 / M00 - x_centroid * y_centroid
11 u20 = M20 / M00 - x_centroid**2
12 u02 = M02 / M00 - y_centroid**2
13 theta = 0.5 * np.arctan2(2 * u11, u20 - u02) # in radians
14 print("M00: ", M00, " M10: ", M10, " M01: ", M01)
15 print("M11 = ", M11, " M20 = ", M20, " M02 = ", M02)
16 print("Area: ", M00, " pixels")
17 print("Centroid: ", (x_centroid, y_centroid), "approx", (int(
    x_centroid), int(y_centroid)))
18 print("Orientation angle (degrees): ", np.degrees(theta))
```

Kết quả

```
M00: 18 M10: 54 M01: 62
M11 = 186 M20 = 210 M02 = 238
Area: 18 pixels
Centroid: (np.float64(3.0), np.float64(3.4444444444444446)) ≈ (3, 3)
Orientation angle (degrees): 0.0
```

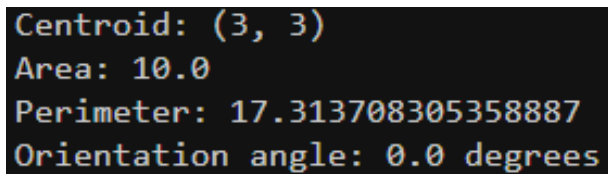
Hình 1.1: Kết quả lập trình tính toán các thông số

Đoạn code sử dụng hàm OpenCV để kiểm tra kết quả

```
1 contours, _ = cv2.findContours(img, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
```

```
2 for contour in contours:
3     # Centroid
4     M = cv2.moments(contour)
5     if M["m00"] != 0:
6         cx = int(M["m10"] / M["m00"])
7         cy = int(M["m01"] / M["m00"])
8         print(f"Centroid: ({cx}, {cy})")
9
10    # Area
11    area = cv2.contourArea(contour)
12    print(f"Area: {area}")
13
14    # Perimeter
15    perimeter = cv2.arcLength(contour, True)
16    print(f"Perimeter: {perimeter}")
17
18    # Orientation
19    if len(contour) >= 5:
20        ellipse = cv2.fitEllipse(contour)
21        angle = ellipse[2]
22        print(f"Orientation angle: {angle} degrees")
```

Kết quả



```
Centroid: (3, 3)
Area: 10.0
Perimeter: 17.313708305358887
Orientation angle: 0.0 degrees
```

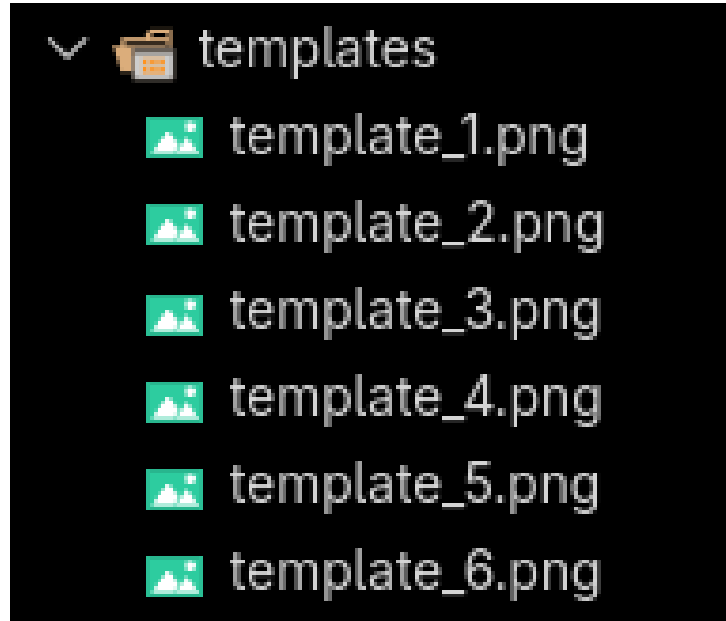
Hình 1.2: Kết quả sử dụng hàm OpenCV để kiểm tra

2 Question 2

2.1 Tìm vị trí các máy bay trong hình bằng phương pháp genral hough transform tự lập trình

2.1.1 Các bước thực hiện

- Tạo các Template của các máy bay trong hình.



Hình 2.1: Các Template của các máy bay trong hình

- Load Template và Scene:

```
1 tpl = cv2.imread(TEMPLATE_PATH, cv2.IMREAD_GRAYSCALE)
2 scene = cv2.imread(SCENE_PATH)
3 scene_gray = cv2.cvtColor(scene, cv2.COLOR_BGR2GRAY)
```

- Xây R-table - với GHT ta lưu cho mỗi hướng biên (edge orientation bin) một hoặc nhiều vector chỉ từ điểm biên tới tham chiếu (center) của template. Khi dò trong ảnh scene, từ mỗi điểm biên cộng vector này sẽ ước lượng vị trí center ứng viên.

```
1 def build_rtable(template_gray):
2     _, bin_img = cv2.threshold(template_gray, 0, 255, cv2.
3     THRESH_BINARY + cv2.THRESH_OTSU)
4     edges = cv2.Canny(bin_img, CANNY_LOW, CANNY_HIGH)
5     gx = cv2.Sobel(template_gray, cv2.CV_32F, 1, 0, ksize
6     =3)
7     gy = cv2.Sobel(template_gray, cv2.CV_32F, 0, 1, ksize
8     =3)
9     ys, xs = np.where(bin_img > 0)
10    if len(xs) == 0:
```

```

10         raise ValueError("Template was empty after
           thresholding.")
11     cx, cy = xs.mean(), ys.mean()
12
13     ys_e, xs_e = np.where(edges > 0)
14
15     # Vectorized: Compute orientations for all edge points
16     theta = np.arctan2(gy[ys_e, xs_e], gx[ys_e, xs_e])
17     theta = np.degrees(theta) % 360.0
18     bin_ang = (theta // ANGLE_BIN).astype(int) * ANGLE_BIN
19
20     rx = cx - xs_e
21     ry = cy - ys_e
22
23     # Create R-table bins
24     n_bins = 360 // ANGLE_BIN
25     rtable_bins = np.arange(0, 360, ANGLE_BIN)
26
27     # Build R-table index
28     rtable_idx = {}
29     for b in rtable_bins:
30         mask = bin_ang == b
31         if np.any(mask):
32             rtable_idx[b] = np.where(mask)[0]
33
34     # Stack r vectors
35     r_vectors = np.column_stack([rx, ry]) # shape: (
           N_edges, 2)
36
37     return rtable_idx, r_vectors, (cx, cy)

```

- Thực hiện voting cho tất cả rotation và scale, tạo accumulator 4D: (H, W, n_scales, n_rotations). Mỗi vote tăng 1 tại vị trí center (x,y) tương ứng scale và rotation.

```

1  def vote_with_rot_scale_vectorized(scene_gray, rtable_idx,
   r_vectors):
2      edges = cv2.Canny(scene_gray, CANNY_LOW, CANNY_HIGH)
3      gx = cv2.Sobel(scene_gray, cv2.CV_32F, 1, 0)
4      gy = cv2.Sobel(scene_gray, cv2.CV_32F, 0, 1)
5
6      H, W = scene_gray.shape
7      n_rots = len(ROTATIONS)
8      n_scales = len(SCALES)
9
10     # Create 4D accumulator: [H, W, n_scales, n_rots]
11     acc_4d = np.zeros((H, W, n_scales, n_rots), dtype=np.
           int32)
12
13     ys_scene, xs_scene = np.where(edges > 0)
14     n_scene = len(xs_scene)
15
16     if n_scene == 0:

```

```

17         return acc_4d
18
19     # Scene angles for all edge pixels
20     theta_scene = np.arctan2(gy[ys_scene, xs_scene], gx[
        ys_scene, xs_scene])
21     theta_scene = np.degrees(theta_scene) % 360.0 # shape:
        (n_scene,)
22
23     print(f"Processing {n_scene} scene edges...")
24
25     # Strategy: Process in batches of (rotation, scale)
        combinations
26
27     for rot_idx in range(n_rots):
28         phi = ROTATIONS[rot_idx]
29         cos_phi = ROT_COS[rot_idx]
30         sin_phi = ROT_SIN[rot_idx]
31
32         # Template angles corresponding to scene angles
        when rotated by phi
33         theta_tpl = (theta_scene - phi) % 360.0 # (n_scene
            ,)
34         bin_tpl = ((theta_tpl // ANGLE_BIN).astype(int) *
            ANGLE_BIN) % 360
35
36         # Rotate ALL r_vectors at once
37         # r_rotated shape: (n_template_edges, 2)
38         r_rotated = np.empty_like(r_vectors)
39         r_rotated[:, 0] = r_vectors[:, 0] * cos_phi -
            r_vectors[:, 1] * sin_phi
40         r_rotated[:, 1] = r_vectors[:, 0] * sin_phi +
            r_vectors[:, 1] * cos_phi
41
42         for scale_idx, scale in enumerate(SCALES):
43             # Scale ALL rotated vectors
44             r_scaled = r_rotated * scale # (
                n_template_edges, 2)
45
46             # For each bin, vote ALL combinations at once
47             for bin_val, template_indices in rtable_idx.
                items():
48                 # Get scene edges belonging to this bin
49                 scene_mask = bin_tpl == bin_val
50                 scene_indices = np.where(scene_mask)[0]
51
52                 if len(scene_indices) == 0:
53                     continue
54
55                 # Get scene edge coordinates
56                 xs_s = xs_scene[scene_indices] # (n_s,)
57                 ys_s = ys_scene[scene_indices] # (n_s,)

```

```

58
59         # Get r vectors from template for this bin
60         r_bin = r_scaled[template_indices] # (n_t,
61         2)
62
63         # VECTORIZED OUTER SUM:
64         # Each scene edge (xs_s[i], ys_s[i]) + each
65         r_bin[j]
66         # Create mesh: (n_s, n_t, 2)
67         xs_mesh = xs_s[:, None] + r_bin[:, 0][None,
68         :] # (n_s, n_t)
69         ys_mesh = ys_s[:, None] + r_bin[:, 1][None,
70         :] # (n_s, n_t)
71
72         # Round and flatten
73         xi = np.round(xs_mesh).astype(int).ravel()
74         yi = np.round(ys_mesh).astype(int).ravel()
75
76         # Filter valid
77         valid = (xi >= 0) & (xi < W) & (yi >= 0) &
78         (yi < H)
79         xi = xi[valid]
80         yi = yi[valid]
81
82         # Vote using np.add.at (atomic add)
83         np.add.at(acc_4d, (yi, xi, scale_idx,
84         rot_idx), 1)
85
86     return acc_4d
  
```

- Tìm vị trí đỉnh trên accumulator (peak detection) theo NMS.

```

1  def find_peaks_vectorized(acc_4d):
2      max_val = acc_4d.max()
3      if max_val == 0:
4          return []
5
6      # Normalize
7      acc_norm = acc_4d.astype(np.float32) / max_val
8
9      # Apply threshold
10     acc_thresh = acc_norm * (acc_norm >= ACC_REL_THRESH)
11
12     # Non-maximum suppression in 4D
13     # Apply local maximum filter
14     footprint = np.ones((5, 5, 1, 1)) # spatial NMS only
15     local_max = maximum_filter(acc_thresh, footprint=
16     footprint, mode='constant')
17
18     # Peaks are where values equal local max
19     peaks_mask = (acc_thresh == local_max) & (acc_thresh >
20     0)
  
```

```

19
20     # Get peak coordinates
21     coords = np.argwhere(peaks_mask) # (N_peaks, 4)
22     scores = acc_thresh[peaks_mask]
23
24     # Sort theo score
25     sort_idx = np.argsort(-scores)[:MAX_PEAKS]
26
27     peaks = []
28     for idx in sort_idx:
29         y, x, scale_idx, rot_idx = coords[idx]
30         score = scores[idx]
31         peaks.append((int(x), int(y), float(score),
32                      float(SCALES[scale_idx]), int(
33                          ROTATIONS[rot_idx])))
34
35     return peaks
    
```

- Tạo bản sao của ảnh gốc scene, sau đó vẽ lên đó vị trí các peak mà GHT tìm được—nghĩa là những chỗ được dự đoán là tâm (center) của template.

```

1 vis = scene.copy()
2     for (x, y, s, scale, rot) in peaks:
3         cv2.circle(vis, (int(x), int(y)), 20, (0, 255, 0),
4                     2)
5         cv2.putText(vis, f"{s:.2f} s={scale:.2f} r={rot}deg",
6                     (int(x)-40, int(y)-25),
7                     cv2.FONT_HERSHEY_SIMPLEX, 0.5,
8                     (0,255,0), 2)
9
10    cv2.imwrite(OUT_PATH, vis)
11    print("Saved to:", OUT_PATH)
    
```

2.1.2 Kết quả thu được

- Với template 1:



Hình 2.2: Template 1

Ta chọn các thông số như sau:

```

1 CANNY_LOW, CANNY_HIGH = 40, 120
2 ANGLE_BIN = 10
3 ACC_REL_THRESH = 0.5
4 MAX_PEAKS = 3
5
6 # rotation & scale search space
7 ROT_STEP = 3
8 SCALE_MIN, SCALE_MAX = 0.5, 1.5
9 SCALE_STEPS = 3
    
```



Hình 2.3: Kết quả tìm kiếm máy bay trong hình với Template 1

- Với template 2:



[H]

Hình 2.4: Template 2

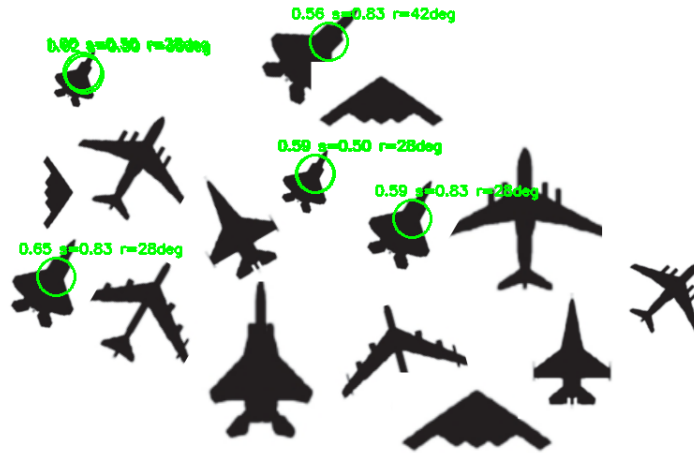
Ta chọn các thông số như sau:

```

1 CANNY_LOW, CANNY_HIGH = 40, 120
2 ANGLE_BIN = 8
3 ACC_REL_THRESH = 0.3
4 MAX_PEAKS = 7
5
    
```

```

6 # rotation & scale search space
7 ROT_STEP = 7
8 SCALE_MIN, SCALE_MAX = 0.5, 1.5
9 SCALE_STEPS = 7
    
```



Hình 2.5: Kết quả tìm kiếm máy bay trong hình với Template 2

- Với template 3:

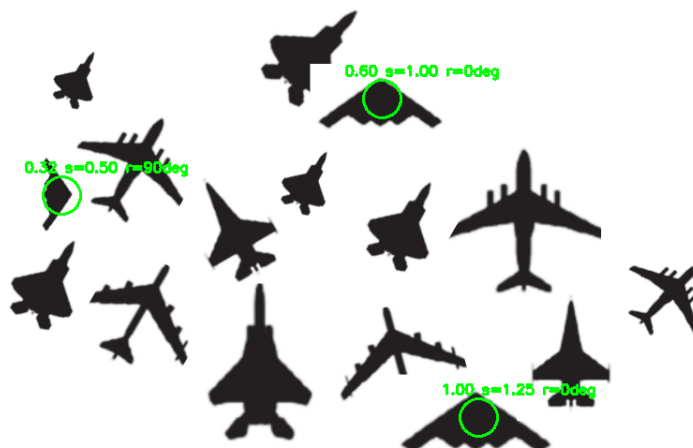


Hình 2.6: Template 3

Ta chọn các thông số như sau:

```

1 CANNY_LOW, CANNY_HIGH = 40, 120
2 ANGLE_BIN = 8
3 ACC_REL_THRESH = 0.3
4 MAX_PEAKS = 3
5
6 # rotation & scale search space
7 ROT_STEP = 5
8 SCALE_MIN, SCALE_MAX = 0.5, 1.5
9 SCALE_STEPS = 5
    
```



Hình 2.7: Kết quả tìm kiếm máy bay trong hình với Template 3

- Với template 4:

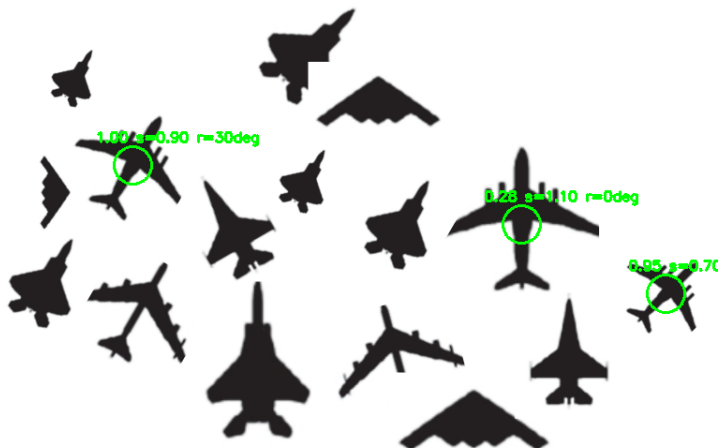


Hình 2.8: Template 4

Ta chọn các thông số như sau:

```

1 CANNY_LOW, CANNY_HIGH = 40, 120
2 ANGLE_BIN = 8
3 ACC_REL_THRESH = 0.2
4 MAX_PEAKS = 3
5
6 # rotation & scale search space
7 ROT_STEP = 5
8 SCALE_MIN, SCALE_MAX = 0.5, 1.5
9 SCALE_STEPS = 6
    
```



Hình 2.9: Kết quả tìm kiếm máy bay trong hình với Template 4

- Với template 5:



Hình 2.10: Template 5

Ta chọn các thông số như sau:

```

1 CANNY_LOW, CANNY_HIGH = 40, 120
2 ANGLE_BIN = 8
3 ACC_REL_THRESH = 0.3
4 MAX_PEAKS = 2
5
6 # rotation & scale search space
7 ROT_STEP = 5
8 SCALE_MIN, SCALE_MAX = 0.5, 1.5
9 SCALE_STEPS = 5
    
```



Hình 2.11: Kết quả tìm kiếm máy bay trong hình với Template 5

- Với template 6:



Hình 2.12: Template 6

Ta chọn các thông số như sau:

```

1 CANNY_LOW, CANNY_HIGH = 40, 120
2 ANGLE_BIN = 8
3 ACC_REL_THRESH = 0.3
4 MAX_PEAKE = 1
5
6 # rotation & scale search space
7 ROT_STEP = 5
8 SCALE_MIN, SCALE_MAX = 0.5, 1.5
9 SCALE_STEPS = 5
    
```



Hình 2.13: Kết quả tìm kiếm máy bay trong hình với Template 6

2.2 Tìm vị trí các máy bay trong hình bằng phương pháp genral hough transform của OpenCV

2.2.1 Các bước thực hiện

- Tạo template các máy bay trong hình và load template cùng scene như trên:
- Detect edges và khởi tạo Ballard:

```
1 scene_edges = cv2.Canny(scene_gray, CANNY_LOW, CANNY_HIGH)
2
3 # Create Ballard
4 ght = cv2.createGeneralizedHoughBallard()
5 ght.setCannyLowThresh(CANNY_LOW)
6 ght.setCannyHighThresh(CANNY_HIGH)
7 ght.setMinDist(MIN_DIST)
8 ght.setVotesThreshold(VOTE_THRESH)
```

- Khởi tạo GHT Ballard và vòng lặp tìm kiếm. Ballard không tự xoay/scale → ta tạo nhiều phiên bản template (xoay + scale), truyền từng cái vào Ballard (setTemplate) rồi gọi detect.

```
1 for scale in SCALES:
2     for angle in ROTATIONS:
3
4         # generate transformed template
5         tpl_rs = rotate_and_scale(tpl0, angle, scale)
6         th, tw = tpl_rs.shape[:2]
7         if th < 5 or tw < 5:
8             continue
9         if th >= Hs or tw >= Ws:
10            continue
11
12        tpl_edges = cv2.Canny(tpl_rs, CANNY_LOW,
13                               CANNY_HIGH)
```

```

14         # Set template for Ballard
15         ght.setTemplate(tpl_edges)
16
17         # Detect, returns (positions, votes)
18         pos, votes = ght.detect(scene_edges)
19
20         if pos is None or len(pos) == 0:
21             continue
22
23         pos = np.array(pos).reshape(-1, 2) # Ballard
24         # returns only (x,y)
25         votes = np.array(votes).reshape(-1)
26
27         for (x, y), v in zip(pos, votes):
28             detections.append({
29                 "x": int(x - tw//2),
30                 "y": int(y - th//2),
31                 "w": tw,
32                 "h": th,
33                 "vote": int(v),
34                 "scale": float(scale),
35                 "angle": float(angle)
36             })

```

- Lọc và hiển thị kết quả:

```

1  filtered = nms(detections, thresh=0.3)
2  vis = scene_color.copy()
3  for d in filtered:
4      x, y, w, h = d["x"], d["y"], d["w"], d["h"]
5      cv2.rectangle(vis, (x, y), (x+w, y+h), (0,255,0),
6                    2)
7      cv2.putText(vis,
8                  f"v={d['vote']} s={d['scale']:.2f} a={d['angle']}",
9                  (x, max(0,y-10)),
10                 cv2.FONT_HERSHEY_SIMPLEX, 0.5,
11                 (0,255,0), 2)
12  cv2.imwrite(OUT_PATH, vis)
13  print("Saved result to:", OUT_PATH)
14
15  try:
16      cv2.imshow("GHT Ballard", vis)
17      cv2.waitKey(0)
18  except:
19      pass

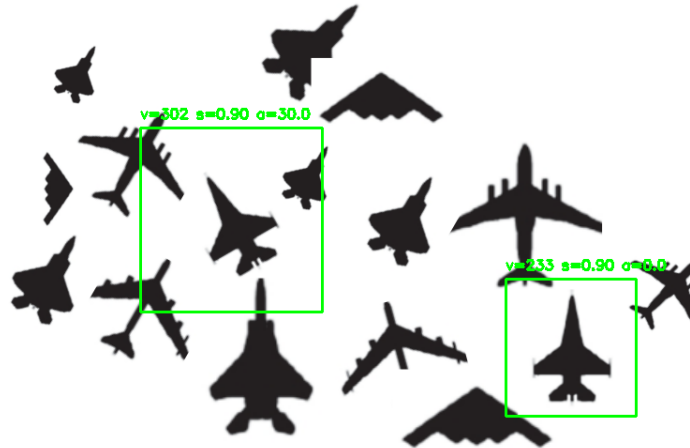
```

2.2.2 Kết quả thu được

- Với template 1: Ta chọn các thông số như sau:

```

1  ANGLE_STEP = 5
2  SCALE_MIN, SCALE_MAX = 0.6, 1.4
3  SCALE_STEPS = 9
4  CANNY_LOW = 40
5  CANNY_HIGH = 120
6  MIN_DIST = 20
7  VOTE_THRESH = 200
8  MAX_FINAL = 2
    
```

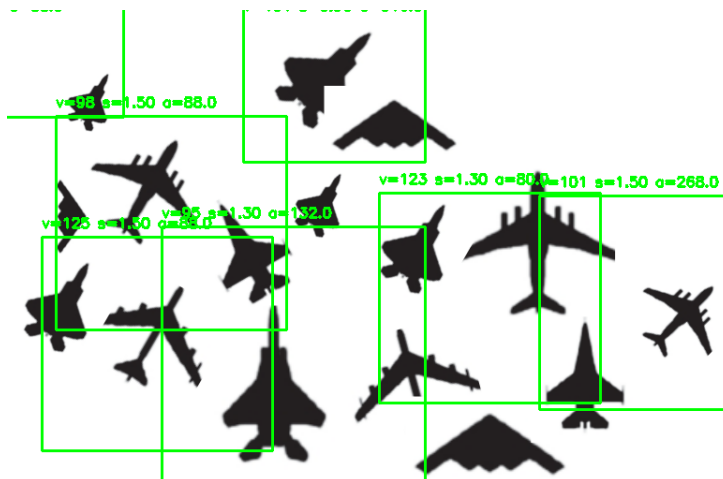


Hình 2.14: Kết quả tìm kiếm máy bay trong hình với Template 1 bằng OpenCV GHT Ballard

- Với template 2: Ta chọn các thông số như sau:

```

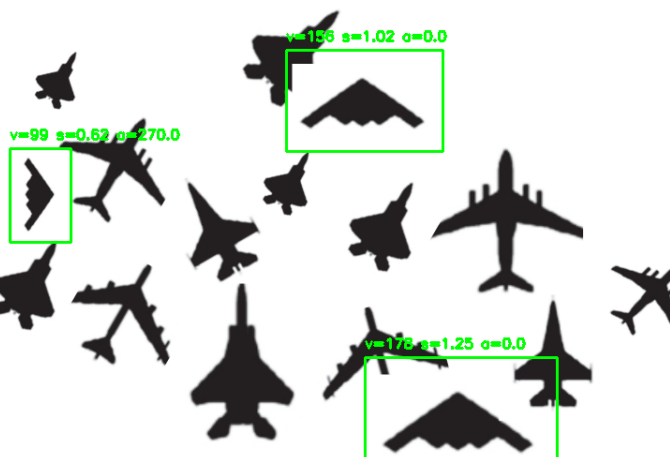
1  ANGLE_STEP = 4
2  SCALE_MIN, SCALE_MAX = 0.5, 1.5
3  SCALE_STEPS = 6
4  CANNY_LOW = 40
5  CANNY_HIGH = 120
6  MIN_DIST = 40
7  VOTE_THRESH = 80
8  MAX_FINAL = 7
    
```



Hình 2.15: Kết quả tìm kiếm máy bay trong hình với Template 2 bằng OpenCV GHT Ballard

- Với template 3: Ta chọn các thông số như sau:

```
1  ANGLE_STEP = 6
2  SCALE_MIN, SCALE_MAX = 0.5, 1.25
3  SCALE_STEPS = 14
4  CANNY_LOW = 40
5  CANNY_HIGH = 120
6  MIN_DIST = 50
7  VOTE_THRESH = 90
8  MAX_FINAL = 3
```



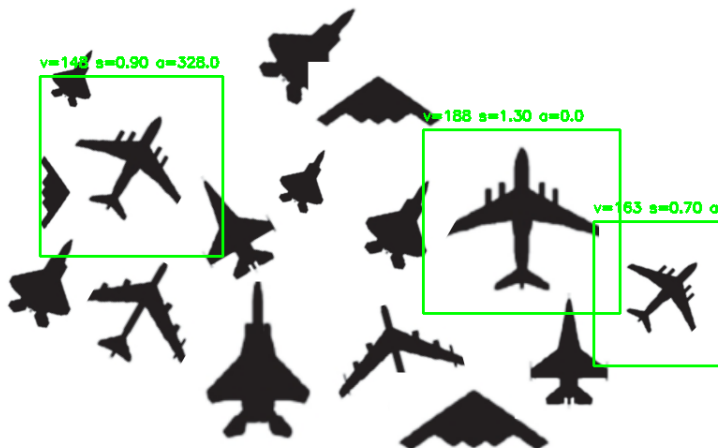
Hình 2.16: Kết quả tìm kiếm máy bay trong hình với Template 3 bằng OpenCV GHT Ballard

- Với template 4: Ta chọn các thông số như sau:

```
1  ANGLE_STEP = 4
2  SCALE_MIN, SCALE_MAX = 0.5, 1.5
3  SCALE_STEPS = 6
4  CANNY_LOW = 40
```

```

5 CANNY_HIGH = 120
6 MIN_DIST = 40
7 VOTE_THRESH = 80
8 MAX_FINAL = 3
    
```

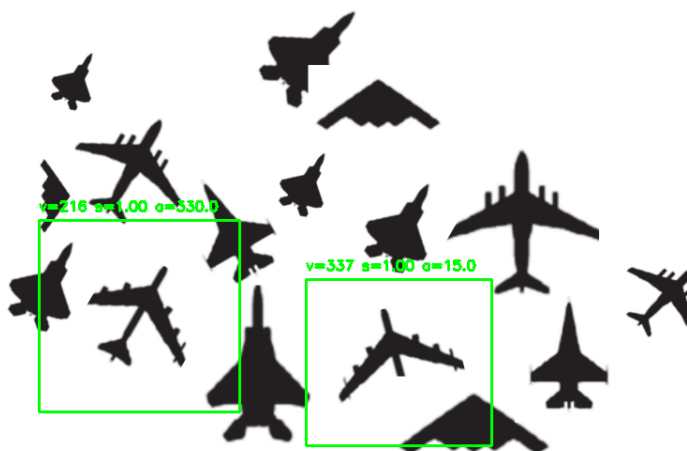


Hình 2.17: Kết quả tìm kiếm máy bay trong hình với Template 4 bằng OpenCV GHT Ballard

- Với template 5: Ta chọn các thông số như sau:

```

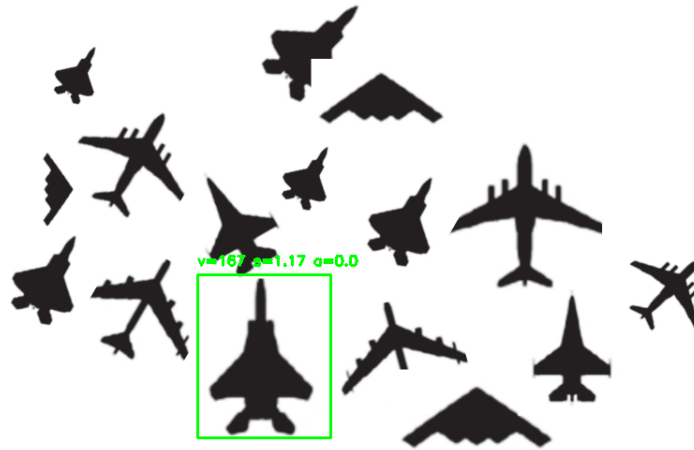
1 ANGLE_STEP = 5
2 SCALE_MIN, SCALE_MAX = 0.5, 1.5
3 SCALE_STEPS = 7
4 CANNY_LOW = 40
5 CANNY_HIGH = 120
6 MIN_DIST = 40
7 VOTE_THRESH = 80
8 MAX_FINAL = 2
    
```



Hình 2.18: Kết quả tìm kiếm máy bay trong hình với Template 5 bằng OpenCV GHT Ballard

- Với template 6: Ta chọn các thông số như sau:

```
1 ANGLE_STEP = 5
2 SCALE_MIN, SCALE_MAX = 0.5, 1.5
3 SCALE_STEPS = 7
4 CANNY_LOW = 40
5 CANNY_HIGH = 120
6 MIN_DIST = 40
7 VOTE_THRESH = 80
8 MAX_FINAL = 1
```



Hình 2.19: Kết quả tìm kiếm máy bay trong hình với Template 6 bằng OpenCV GHT Ballard

2.3 Nhận xét

- Kết quả thu được từ cả hai phương pháp GHT tự lập trình và GHT OpenCV đều khá tốt, có thể phát hiện được hầu hết các máy bay trong hình với các template khác nhau.
- Cả 2 phương pháp đều cần lập trình thêm phần xoay và scale cho template để có thể tìm kiếm đa dạng các đối tượng trong hình cùng với việc lọc kết quả bằng NMS để giữ lại các đỉnh tốt nhất.
- GHT OpenCV cung cấp một giải pháp nhanh chóng và tiện lợi, tuy nhiên việc thiếu khả năng tùy chỉnh sâu có thể hạn chế hiệu suất trong một số trường hợp cụ thể, ví dụ đối với trường hợp template số 2, GHT OpenCV xử lý chưa được tốt do không thể can thiệp quá sâu vào bên trong thuật toán.
- GHT tự lập trình cho phép kiểm soát chi tiết hơn về quá trình, linh động trong việc điều chỉnh các tham số và tối ưu hóa thuật toán cho các trường hợp cụ thể. Tuy nhiên, việc này đòi hỏi nhiều thời gian và công sức để phát triển và tinh chỉnh.
- Cả hai phương pháp đều yêu cầu lựa chọn tham số cẩn thận (như ngưỡng Canny, bước góc, phạm vi tỉ lệ) để đạt được kết quả tốt nhất.